

Platform Engineer

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 23: AI

1. What Platform Engineers Build and Operate

Platform Engineers design, provision, and operate the infrastructure layer that every AI workload depends on. In an AI-enabled organization this scope is considerably wider than traditional cloud infrastructure: it includes GPU compute clusters (NVIDIA A100 and H100), high-throughput distributed file systems for training datasets, object storage for model artifacts, Kubernetes-based container orchestration, VPCs, service meshes, and the API-gateway layer through which AI services are exposed. Platform Engineers are not responsible for model quality or business logic — but every security decision they make (or fail to make) becomes the environment in which model training, inference, and monitoring run. When Platform Engineering gets it wrong, the entire AI stack above it is compromised, regardless of how well-secured the application layer appears.

The key insight for SecAI+ candidates: Platform Engineers configure the environment. Their misconfigurations are not isolated — they create the baseline vulnerabilities that adversaries exploit to reach model artifacts, training datasets, and inference APIs. A misconfigured Kubernetes RBAC policy is not a Platform Engineering problem alone; it is an AI security problem for every system running on that cluster.

2. Kubernetes Security for AI Workloads

Kubernetes is the dominant orchestration platform for AI workloads, and its security posture is a direct reflection of Platform Engineering discipline. The most critical Kubernetes misconfiguration categories for AI environments are summarized in the table below. The NSA/CISA Kubernetes Hardening Guide and CIS Kubernetes Benchmark provide the configuration baselines Platform Engineers must implement before any AI workloads are scheduled.

Misconfiguration Category	AI-Specific Risk	Key Remediation
Overly permissive RBAC	Service account can read all Secrets — API keys and model credentials exposed	Least-privilege roles; bind only required verbs and resources
Pods running as root	Container escape grants root on the host node; GPU memory accessible	SecurityContext: runAsNonRoot: true; drop ALL capabilities
Unencrypted etcd	Cluster state (including Secrets) readable by anyone with etcd access	Enable etcd encryption at rest using AES-CBC or AES-GCM provider
Exposed Kubernetes API server	Unauthenticated API access enables cluster takeover	Restrict API server to private network; enforce mutual TLS
Permissive NetworkPolicies	Training pods communicate laterally — pivoting between tenants	Deny-all baseline; allow only required pod-to-pod paths
GPU sharing without isolation	Cross-tenant GPU memory access via shared GPU instances	Use NVIDIA MIG to isolate GPU partitions per tenant

3. Container Security: Images, Registries, and Runtime

AI training containers are long-running, GPU-accelerated, and often require broad network access to download datasets and publish results — a combination that makes them attractive targets for compromise and difficult to monitor effectively. Container security for AI workloads spans three phases.

- **Build phase — Image scanning:** Tools such as Trivy, Grype, and Snyk Container scan container images for known CVEs in base images and Python dependencies before the image is pushed to the registry. AI images commonly include PyTorch, TensorFlow, CUDA drivers, and dozens of ML framework libraries — each a potential vulnerability surface.
- **Distribute phase — Registry security and image signing:** Container registries (Harbor, AWS ECR, Azure Container Registry) should enforce image signing using Cosign or Notary. Only signed images with verified provenance should be schedulable on production clusters. Unsigned or unverified base images from public Docker Hub are a supply chain risk.
- **Runtime phase — Behavioral monitoring:** Falco monitors container syscalls at runtime and generates alerts when a container deviates from expected behavior — attempting to write to restricted filesystem paths, spawn unexpected processes, or establish outbound network connections to unrecognized destinations. AI training containers that suddenly initiate external DNS lookups or persistent outbound connections are exhibiting indicators of compromise.

Exam tip: Runtime security tools like Falco operate at the kernel syscall level — they detect anomalous behavior that cannot be caught at the image-scan stage because it only manifests at execution time. Know that Falco is a runtime tool, not a scanner.

4. Real-World Incident: Compromised GPU Instances (2023)

Real-World Incident — Exposed Jupyter Notebook Interfaces (2023): Researchers identified multiple AWS EC2 GPU instances provisioned for AI training that had been compromised through Jupyter Notebook interfaces running without authentication on public network interfaces. Attackers used the GPU compute for cryptocurrency mining — a highly motivated objective given GPU scarcity and cost. Some compromised instances also had access to training datasets and model artifacts in attached S3 buckets, creating an intellectual property theft risk. The root cause was a Platform Engineer misconfiguration: Jupyter was started in testing mode (binding to 0.0.0.0 without a token) and the configuration was never hardened before the instance joined the production subnet.

The corrective controls Platform Engineers must implement include: (1) network policies preventing training infrastructure from being directly reachable from the public internet; (2) regular audit of listening services on training hosts using tools such as Shodan or cloud-native service discovery; (3) mandatory authentication for any management interface, including Jupyter, MLflow Tracking Server, and Ray Dashboard. This incident demonstrates that GPU infrastructure attacks are motivated not only by espionage but by the economic value of GPU compute itself.

5. Secrets Management for AI Workloads

AI training jobs are secret-hungry: they need database credentials, API keys for external data sources, cloud storage access keys, experiment-tracking tokens (W&B, Neptune), and Hugging Face Hub access tokens for downloading model weights. Hardcoded secrets in container images or Kubernetes manifests checked into source control are a perennial finding in AI platform security assessments.

- HashiCorp Vault: An open-source secrets management system that provides dynamic secrets (credentials generated on demand and auto-revoked after TTL), secret leasing, and an audit log of every secret access. AI training pipelines can use Vault's Kubernetes auth method to obtain credentials at pod startup without storing static secrets anywhere.
- AWS Secrets Manager / Azure Key Vault: Cloud-native managed services that integrate directly with cloud IAM. Kubernetes External Secrets Operator retrieves secrets from these services at pod creation time and injects them as Kubernetes Secrets — never persisted in Git or container images.
- Sealed Secrets (Bitnami): Encrypts Kubernetes Secrets with the cluster's public key so they can be safely committed to Git. The cluster controller decrypts them — plaintext secrets exist only inside the cluster.
- **NIST CSF 2.0 mapping:** Secrets management maps to the PR.AC (Identity Management, Authentication, and Access Control) subcategory. AI components authenticating with managed, rotated credentials rather than hardcoded secrets satisfy this control.

6. Multi-Tenancy and Resource Isolation

Organizations running shared AI platforms for multiple teams or business units must implement multi-tenancy: the ability to run multiple teams' workloads on shared infrastructure without allowing cross-tenant access to data, GPU memory, or credentials. Multi-tenancy failures in AI platforms are particularly serious because they can expose proprietary model training data across business unit or client boundaries — a data breach that may not leave obvious logs and may not be discoverable until an adversary exploits it.

Isolation Layer	Mechanism	AI-Specific Concern
Namespace isolation	Kubernetes Namespaces + RBAC	Prevents Team A's service accounts from reading Team B's Secrets or PVCs
Network isolation	Kubernetes NetworkPolicies	Blocks cross-namespace training pod communication; prevents lateral movement
Compute isolation	ResourceQuotas + LimitRanges	Prevents one team's training run from exhausting cluster CPU/memory/GPU
GPU isolation	NVIDIA MIG (Multi-Instance GPU)	Partitions GPU hardware so training jobs cannot access each other's GPU memory
Data access isolation	IAM role per namespace + bucket policies	Ensures training datasets are accessible only to the authorized team's service accounts

7. Observability Infrastructure for Security

Platform Engineers build the observability stack that serves both operational and security purposes. The security value of a well-designed observability stack is that it provides the telemetry necessary for threat detection in AI workloads — anomalies that would otherwise be invisible become detectable signals.

- **Metrics collection:** Prometheus scrapes metrics from Kubernetes nodes, GPU exporters (DCGM Exporter for NVIDIA GPUs), and AI application endpoints. Grafana visualizes these metrics. Anomalous GPU utilization — sustained 100% utilization from a pod not scheduled for training — can indicate cryptomining.
- **Log aggregation:** The ELK Stack (Elasticsearch, Logstash, Kibana) or Datadog aggregates logs from training pods, inference servers, and pipeline orchestrators. Log anomalies in inference services — unexpected request patterns, error spikes — may indicate adversarial probing.
- **Distributed tracing:** Jaeger or Zipkin traces requests across microservices. In AI inference architectures with multiple model-serving components, distributed tracing identifies which component is generating anomalous latency or errors.
- **NIST CSF 2.0 mapping:** Security-oriented observability implements DE.AE (Adverse Event Analysis). Platform Engineers who design observability with security use cases in mind — not just operational dashboards — directly contribute to the organization's threat detection capability.

8. Storage Security for Training Data and Model Artifacts

Training datasets and model artifacts are high-value intellectual property. A compromised or tampered artifact store is an AI supply chain attack: if a model file is replaced between training evaluation and production deployment, the organization deploys a model it has not validated. Platform Engineers must implement storage controls that guarantee confidentiality, integrity, and access auditability.

Control	Mechanism	Risk Addressed
Encryption at rest	SSE-KMS (customer-managed keys) on S3/Blob storage	Dataset and model artifact confidentiality if storage is exposed
Access control	IAM roles with least privilege; bucket policies	Unauthorized read or write of training data or model weights
Versioning	S3 Object Versioning or Azure Blob Versioning	Preserves dataset and artifact history; enables rollback after tampering
Access logging	S3 Server Access Logs / Azure Storage Analytics	Audit trail: who read or wrote which dataset or model artifact, and when
Artifact integrity	Checksum verification + signed artifact stores	Prevents supply chain attack via model artifact replacement between evaluation and deployment

9. Network Security and AI Traffic Patterns

AI workloads generate traffic patterns that differ substantially from typical web application workloads, and Platform Engineers must design network architecture around these patterns — not adapt AI workloads to fit a network designed for conventional web applications.

- Distributed training (east-west): NCCL (NVIDIA's collective communications library) generates very high-bandwidth, low-latency east-west traffic between GPU nodes during distributed training. This traffic must be on isolated, trusted network segments — it should never traverse public-facing interfaces or shared networks where other tenants operate.
- Model inference (north-south): Inference APIs receive client requests and return model predictions. This traffic should traverse API gateways with authentication, rate limiting, and request logging before reaching the model-serving layer. Unauthenticated inference endpoints enable model extraction attacks.
- Data pipeline traffic: Traffic between data stores and training infrastructure should be encrypted in transit (TLS 1.2 minimum) and flow through dedicated network segments with firewall rules permitting only the specific source-destination pairs required.
- Management interfaces: Jupyter Notebook, MLflow Tracking Server, Ray Dashboard, and Kubeflow Pipelines Dashboard must be accessible only from authorized management network segments — never from public interfaces.

10. Red Team Perspective: Infrastructure-Level Attacks on AI Systems

Red teams targeting AI infrastructure focus on the platform layer because compromises at this level sit below all application-layer security controls. An attacker who owns the Kubernetes cluster owns every AI system running on it, regardless of how well-secured those applications are individually.

- Exposed management interfaces: Shodan and similar tools can enumerate publicly accessible Jupyter Notebook, MLflow, and Ray Dashboard instances. These interfaces often provide direct code execution in the context of the service account running the workload.
- Service account escalation: RBAC misconfigurations that grant service accounts the ability to create or modify ClusterRoleBindings allow privilege escalation from a compromised training pod to cluster administrator access.
- Unencrypted etcd access: An attacker with network access to the etcd API can extract all Kubernetes Secrets in plaintext, including model registry credentials, cloud provider access keys, and database passwords.
- Container image tampering: If the image registry allows unauthenticated pushes or lacks image signing enforcement, an attacker can replace a training image with a backdoored version that exfiltrates datasets or inserts a poisoning step into the training loop.
- **Defense-in-depth principle:** Platform Engineers who implement defense in depth — network isolation, Kubernetes hardening, runtime monitoring, secrets management — force adversaries to operate above the infrastructure layer where application-layer controls apply. A compromised application is serious; a compromised platform compromises everything.

Analogy: Think of the AI platform as the foundation of a building. Security controls in the application layer (model serving, API authentication, input validation) are the locks on the doors and windows. An attacker who cracks the foundation does not need to pick locks — the entire structure is accessible. Platform security is the foundation that makes every other control meaningful.

Key Terms Glossary

Term	Definition
Kubernetes RBAC	Role-Based Access Control within Kubernetes — defines which subjects (users, service accounts) can perform which verbs (get, list, create, delete) on which resources (pods, secrets, deployments)
etcd	The key-value store used by Kubernetes to persist all cluster state, including Secrets objects. If unencrypted and network-accessible, an adversary can extract all Secrets in plaintext by querying etcd directly
NVIDIA MIG (Multi-Instance GPU)	Multi-Instance GPU partitioning that divides a single physical GPU into multiple isolated instances, each with dedicated memory, preventing cross-tenant GPU memory access
Falco	Open-source runtime security tool that monitors Linux kernel system calls and alerts when container behavior deviates from defined rules — unexpected network connections, writes to sensitive paths, privilege escalation attempts
Trivy	Open-source container image and filesystem scanner that identifies known CVEs in container base images, application dependencies, and infrastructure-as-code configurations
NSA/CISA Kubernetes Hardening Guide	Joint publication providing specific configuration guidance for Kubernetes cluster security covering pod security, RBAC, network policies, audit logging, and secrets management
CIS Kubernetes Benchmark	Configuration security benchmark from the Center for Internet Security providing scored recommendations for Kubernetes cluster hardening
NIST SP 800-190	NIST publication on Application Container Security Guide covering image security, registry security, orchestration platform security, and container runtime security
NCCL	NVIDIA Collective Communications Library — provides optimized communication primitives for distributed GPU training, generating high-bandwidth east-west traffic requiring network isolation
HashiCorp Vault	Open-source secrets management platform providing dynamic secrets, secret leasing, and a comprehensive audit log — widely used for AI training pipeline credential management
DCGM Exporter	NVIDIA Data Center GPU Manager Exporter — exports GPU metrics (utilization, memory usage, temperature) to Prometheus for monitoring AI workload GPU consumption and detecting anomalies
Cosign / Notary	Tools for cryptographically signing container images and verifying signatures at deployment time, ensuring only verified images with documented provenance are scheduled on production clusters

Quick Revision Checklist

- Platform Engineers build and secure GPU clusters, container platforms, storage, and networking — the foundation of every AI system.

- Kubernetes security essentials: RBAC least-privilege, pod security contexts (non-root), etcd encryption at rest, API server network restriction, and namespace-level NetworkPolicies.
- Container security spans three phases: build (image scanning with Trivy/Grype), distribute (image signing with Cosign/Notary), and runtime (behavioral monitoring with Falco).
- GPU memory isolation requires NVIDIA MIG partitioning on shared clusters — Kubernetes namespace isolation alone does not prevent cross-tenant GPU memory access.
- Secrets management: never hardcode credentials in images or manifests — use Vault, AWS Secrets Manager, or the Kubernetes External Secrets Operator.
- AI traffic patterns: distributed training (NCCL) is east-west and must be isolated on trusted networks; inference (north-south) must traverse authenticated API gateways; management interfaces must be restricted to private networks.
- The 2023 exposed Jupyter incident demonstrates that management interfaces left in testing mode become permanent attack surfaces enabling both compute abuse (cryptomining) and data exfiltration.
- Key frameworks: NSA/CISA Kubernetes Hardening Guide, CIS Kubernetes Benchmark, NIST SP 800-190 (Application Container Security).
- Key tools: Falco (runtime security), Trivy/Grype (image scanning), Vault/Secrets Manager (secrets), DCGM Exporter (GPU metrics), Cosign/Notary (image signing).
- NIST CSF 2.0 mappings: secrets management maps to PR.AC; security-oriented observability maps to DE.AE.
- Exam tip: Infrastructure-level compromises sit below application-layer controls — defense in depth at the platform layer is the prerequisite for application-layer controls to be effective.

10 Exam-Based MCQs — Platform Engineer

Test yourself after reading. These questions are written in real SecAI+ exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. Scenario: A security review of a financial services firm's AI training platform finds that a Jupyter Notebook instance is accessible from the public internet without authentication. The instance runs on an EC2 GPU node with an IAM role attached that grants read access to training datasets in S3 and write access to the model artifact bucket. Which action should the Platform Engineer take FIRST to remediate the risk?

- A) Install Falco on all training nodes to monitor for anomalous GPU usage
- B) Rotate all IAM credentials used by the training service accounts
- C) Restrict the Jupyter Notebook interface to accept connections only from the internal management subnet
- D) Enable S3 versioning on the training dataset bucket

Answer: C **Explanation:** C is correct because the immediate risk is the publicly exposed Jupyter Notebook interface — an unauthenticated management interface providing direct code execution. Restricting it to the management subnet eliminates the external attack surface before any other remediation is meaningful. A (Falco) is valuable for ongoing monitoring but does not remove the exposure. B (credential rotation) is appropriate as a follow-up once the interface is secured, but rotating credentials

while the interface remains public leaves the attack surface open. D (S3 versioning) is useful for artifact integrity but does not address the direct infrastructure exposure.

Q2. Scenario: A healthcare organization runs an AI training platform on a shared Kubernetes cluster serving three separate business units. A penetration test finds that a training pod from Business Unit A was able to read Kubernetes Secrets belonging to Business Unit B. Which Kubernetes misconfiguration MOST directly enables a cross-tenant data access attack on a shared AI training platform?

- A) Pods running as root with no security context restrictions
- B) Missing resource quotas allowing one training job to exhaust cluster compute
- C) Service account tokens mounted in all pods by default with cluster-wide read access to Secrets
- D) Container images pulled from a public registry without signature verification

Answer: C Explanation: C is correct because service account tokens with cluster-wide read access to Secrets enable any compromised pod to read secrets and credentials belonging to other tenants via the Kubernetes API — directly enabling cross-tenant data access. A (root pods) increases the blast radius of a container escape but does not directly enable cross-tenant data access at the API level. B (missing quotas) causes resource exhaustion — a denial-of-service concern, not a data access concern. D (unsigned images) is a supply chain concern that may introduce malicious code but does not by itself enable cross-tenant access.

Q3. Scenario: A Platform Engineer replaces hardcoded AWS access keys embedded in training job container images with HashiCorp Vault dynamic secrets that are provisioned at pod startup via the Kubernetes auth method and automatically revoked after the training job completes. What NIST CSF 2.0 subcategory does this control MOST directly address?

- A) PR.DS — Data Security
- B) DE.AE — Adverse Event Analysis
- C) PR.AC — Identity Management, Authentication, and Access Control
- D) RS.RP — Incident Management

Answer: C Explanation: C is correct. Replacing hardcoded credentials with dynamically provisioned, auto-rotating credentials managed by HashiCorp Vault directly implements identity management and access control for AI system components — the PR.AC subcategory. A (PR.DS) covers data protection including encryption, relevant to credentials at rest but not the primary subcategory for credential lifecycle management. B (DE.AE) covers adverse event analysis — security monitoring and detection, not credential management. D (RS.RP) covers incident response planning, not preventive access controls.

Q4. A training container on a production AI platform suddenly begins making outbound HTTPS connections to an unrecognized external IP address. Which tool is BEST suited to detect this behavior at runtime?

- A) Trivy
- B) Cosign

- C) Prometheus with DCGM Exporter
- D) Falco

Answer: D **Explanation:** D is correct. Falco monitors Linux kernel system calls at runtime and generates alerts when a container exhibits anomalous behavior — establishing unexpected outbound network connections is precisely the type of behavioral deviation Falco is designed to detect. A (Trivy) is an image scanner that runs before deployment; it has no runtime monitoring capability. B (Cosign) verifies image signatures at deployment time — it has no runtime visibility. C (Prometheus with DCGM Exporter) monitors GPU utilization metrics — it could detect anomalous GPU usage but is not designed to detect specific network connection anomalies from individual containers.

Q5. Which of the following BEST describes the purpose of NVIDIA MIG in an AI platform security context?

- A) Encrypting GPU memory contents to prevent data exfiltration through GPU channels
- B) Partitioning a physical GPU into isolated instances so training jobs cannot access each other's GPU memory
- C) Monitoring GPU utilization for anomalies that may indicate cryptomining on shared infrastructure
- D) Signing GPU compute kernels to prevent injection of malicious code into training computations

Answer: B **Explanation:** B is correct. NVIDIA Multi-Instance GPU (MIG) technology partitions a physical GPU into multiple hardware-isolated instances, each with dedicated memory bandwidth and GPU SM allocation, preventing cross-tenant memory access — this is its primary security value in multi-tenant AI platforms. A (GPU memory encryption) is a feature of NVIDIA's Confidential Computing capability available on H100, which is distinct from MIG partitioning. C (utilization monitoring) is performed by DCGM Exporter, not MIG. D (kernel signing) is not a feature of MIG; this describes a different area of firmware and compute integrity.

Q6. Which of the following is the CORRECT description of etcd's security significance in a Kubernetes cluster running AI workloads?

- A) etcd is the container runtime — if compromised, attackers can execute arbitrary commands in any container
- B) etcd is the Kubernetes API server — if exposed publicly, attackers can manage cluster resources directly
- C) etcd is the key-value store holding all cluster state including Secrets — if unencrypted and accessible, all credentials are exposed in plaintext
- D) etcd is the container image registry — if compromised, attackers can push malicious images to the cluster

Answer: C **Explanation:** C is correct. etcd is Kubernetes' distributed key-value store that persists all cluster configuration and state — including Kubernetes Secrets objects. If etcd is not encrypted at rest and is network-accessible, an adversary can extract all Secrets in plaintext by querying etcd directly, bypassing RBAC controls entirely. A describes the container runtime (containerd, CRI-O), not etcd. B describes the

Kubernetes API server, a separate component. D describes a container registry (Harbor, ECR), not etcd.

Q7. Which frameworks provide the configuration baselines that Platform Engineers should implement for Kubernetes clusters running AI workloads?

- A) MITRE ATLAS v2.1 and OWASP LLM Top 10 v1.1
- B) NSA/CISA Kubernetes Hardening Guide and CIS Kubernetes Benchmark
- C) NIST SP 800-53 Revision 5 and ISO/IEC 27001:2022
- D) OWASP Top Ten 2021 and NIST SP 800-190

Answer: B **Explanation:** B is correct. The NSA/CISA Kubernetes Hardening Guide and CIS Kubernetes Benchmark provide specific, actionable configuration guidance for Kubernetes cluster security — covering pod security, RBAC, network policies, audit logging, and secrets management. The lecture specifically cites these as the baseline frameworks. A (MITRE ATLAS + OWASP LLM Top 10) are adversarial ML and LLM threat frameworks, not Kubernetes configuration standards. C (NIST SP 800-53 + ISO 27001) are general security control and management system standards — they do not provide Kubernetes-specific configuration guidance. D (OWASP Top Ten + NIST SP 800-190) are relevant to web and container security broadly but are not the primary Kubernetes-specific frameworks.

Q8. What is the PRIMARY security value of enabling access logging on S3 buckets used for AI model artifact storage?

- A) Prevents unauthorized users from reading model artifacts by enforcing authentication at the storage layer
- B) Creates an audit trail of who read or wrote which model artifact and when, supporting incident response and supply chain forensics
- C) Encrypts model artifacts in transit between the training cluster and the storage bucket
- D) Enables automatic versioning of model artifacts to allow rollback after a supply chain attack

Answer: B **Explanation:** B is correct. S3 Server Access Logs record every API call to the bucket — requester identity, source IP, timestamp, and resource accessed. This audit trail is essential for incident response (determining scope of unauthorized access) and supply chain forensics (identifying which artifact was read or replaced and by whom). A (preventing unauthorized access) is the function of bucket policies and IAM — not access logging, which is a detective control, not a preventive one. C (encryption in transit) is implemented by enforcing HTTPS on the bucket, not by access logging. D (versioning) is a separate S3 feature from access logging.

Q9. Why MUST distributed training network traffic using NCCL be isolated from public-facing network segments?

- A) NCCL is unencrypted by default, and isolation prevents interception of gradient updates that may contain information about training data
- B) NCCL traffic volume would saturate public internet connections, causing availability failures for external services

- C) Regulatory requirements mandate that all AI training data remain within private networks and cannot traverse internet-connected segments
- D) GPU compute costs increase significantly when NCCL traffic is routed through public NAT gateways

Answer: A **Explanation:** A is correct. NCCL communicates gradient updates between GPU nodes, and these gradients can leak information about training data (gradient inversion attacks can reconstruct training samples from intercepted gradient updates). Additionally, NCCL traffic is typically unencrypted because it runs on trusted internal networks by design. Isolating this traffic to private, trusted network segments prevents interception and potential gradient-based training data reconstruction. B (bandwidth saturation) is operationally relevant but is not the primary security rationale. C (regulatory requirement) may exist in some jurisdictions but is not the general security rationale. D (cost) is an operational concern, not a security rationale.

Q10. Which of the following BEST describes the security risk of pulling AI training container images from public Docker Hub without signature verification?

- A) Container images from Docker Hub are larger than private registry images, increasing attack surface through additional installed packages
- B) Docker Hub does not support GPU acceleration for training containers, requiring attestation before GPU scheduling
- C) A malicious actor could publish a typosquatted or hijacked image containing arbitrary code that exfiltrates datasets or poisons model weights during training
- D) Public images cannot be scanned by Trivy because they are stored outside the organization's network perimeter

Answer: C **Explanation:** C is correct. Without image signature verification, there is no cryptographic guarantee that a pulled image is the authentic version published by the legitimate maintainer. Attackers have published typosquatted images (names similar to popular packages) and have compromised registry accounts to replace legitimate images with backdoored versions. A training container that exfiltrates datasets or inserts poisoning logic is an AI supply chain attack enabled precisely by the absence of signature verification. A (image size) is irrelevant to the described security risk. B is factually incorrect — Docker Hub fully supports GPU-accelerated images. D is incorrect — Trivy can scan public images by pulling them for analysis.